Autonomous Pricing Agent MVP: Comprehensive Technical & Architecture Interview Guide

1. Executive Summary & Project Pitch

The 60-Second Elevator Pitch

The Autonomous Pricing Agent MVP is an end-to-end hybrid AI system that combines gradient-boosted demand forecasting (LightGBM), a constrained non-convex optimization engine with deterministic business guardrails, a FastAPI microservice layer, and an LLM-powered conversational orchestrator built in n8n. Unlike pure LLM systems that hallucinate financial calculations, or unguarded ML models that recommend destructive sub-cost pricing, this architecture separates probabilistic language routing from deterministic mathematical optimization. It ingests 1,067,371 raw retail transaction records, models price elasticity across the top 100 high-velocity SKUs, enforces margin floor and price-velocity constraints, and exposes real-time optimization and what-if simulation tools to an autonomous AI agent.

Dual-Perspective Overview: Why This Architecture Wins in Production

Dimension	Technical Interview Framing (How to Say It to the Panel)	Intuitive Engineering Explanation (How It Works Under the Hood)	CS / AI Domain Analogy
System Paradigm	Neuro-symbolic / Tool-Augmented Agentic Architecture decoupling LLM semantic parsing from deterministic mathematical inference.	The LLM never does math. It translates human questions into structured API calls, while Python and LightGBM handle all demand predictions and profit calculations.	Like a SQL Query Planner vs. an LLM Text-to-SQL translator: the LLM writes the query, but the database engine executes the relational algebra deterministically.
Demand Modeling	Non-parametric point-elasticity estimation via Gradient Boosted Decision Trees (LightGBM) trained on weekly aggregated panel data.	Instead of assuming demand is a straight line, we group transactions by week and train decision trees to learn how unit sales respond to price changes and calendar seasonality.	Like profiling CPU cache hit rates across varying workload sizes: performance drops in non-linear steps rather than a smooth line.

Dimension	Technical Interview Framing (How to Say It to the Panel)	Intuitive Engineering Explanation (How It Works Under the Hood)	CS / AI Domain Analogy
Optimization Solver	Bounded 1D Grid Search over a non-differentiable step-function response surface within a feasible constraint polytope.	Tree models output step-like predictions where calculus derivatives are zero or undefined. Evaluating 50 evenly spaced points across the allowed price range finds the exact peak in milliseconds.	Like hyperparameter tuning via Grid Search over a discrete learning-rate schedule when the validation loss surface is non-differentiable.
Safety Layer	Hard boundary constraint projection enforcing minimum gross margin floors and maximum step-size velocity caps.	Before the solver tests any prices, we clip the search window so it can never recommend selling below a 15% margin or changing the price by more than ±15% at once.	Like gradient clipping and trust-region policy optimization (TRPO) in reinforcement learning, preventing a single update from destabilizing the policy.

---

2. End-to-End System Architecture (The 4-Layer Blueprint)

Layer 1: Data & ML Training Pipeline (`src/1_prep_and_train.py`)

- **Technical Framing**: Offline batch ETL and feature engineering pipeline that transforms raw transactional event logs into a balanced panel dataset for supervised regression, outputting serialized model weights (`demand_model.joblib`), product metadata (`product_catalog.json`), and schema definitions (`feature_cols.json`).
- **Intuitive Explanation**: Reads the 1M+ row CSV, removes bad rows (cancelled orders, negative prices, postage fees, wholesale bulk anomalies), rolls daily receipts up into weekly totals for the top 100 products, trains the LightGBM model on everything except the final 12 weeks, and saves the trained brain to disk.
- **CS/AI Example**: Equivalent to the offline pre-training and indexing phase in a Retrieval-Augmented Generation (RAG) pipeline, where raw documents are chunked, embedded, and written to a vector store so runtime queries don't re-process raw text.

Layer 2: Constrained Optimization & Guardrail Engine (`src/2_solver_and_guardrails.py`)

- **Technical Framing**: Stateless prescriptive analytics engine implementing feasible-set projection (`PricingGuardrails`) and discrete objective maximization (`PricingOptimizer`) over LightGBM demand response curves.
- **Intuitive Explanation**: Loads the saved model and product catalog into memory. When asked for the best price, it calculates the lowest and highest safe prices allowed by business rules, asks the LightGBM model how many units we would sell at 50 price points in between, and picks the price that yields the highest profit or revenue.
- **CS/AI Example**: Similar to constrained decoding or beam search with logit processors in LLM inference, where invalid tokens (violating grammar or safety schemas) are masked out (-inf) before selecting the argmax token.

Layer 3: Microservice & Schema Sanitization Layer (`src/3_api_and_agent.py`)

- **Technical Framing**: Asynchronous ASGI REST service built with FastAPI and Pydantic v2, featuring pre-validation input coercion (`@field_validator(mode="before")`) and CORS middleware to expose idempotent tool endpoints to containerized agents.
- **Intuitive Explanation**: Keeps the ML model loaded in RAM so responses take 10 milliseconds instead of 10 seconds. It also acts as a shock absorber for messy LLM outputs—automatically stripping currency symbols like "£0.50" or translating phrases like "maximize our gross profit" into exact internal enum codes.
- **CS/AI Example**: Acts like an API Gateway with a custom deserialization middleware that normalizes loosely typed JSON payloads from external webhooks before passing strict structs to a gRPC backend.

Layer 4: Agentic Orchestration & Container Stack (n8n + Docker Compose)

- **Technical Framing**: Multi-container Docker Compose deployment coupling a Python 3.11 microservice (`pricing-api`) with a LangChain-backed n8n workflow (`n8n-pricing`), utilizing Window Buffer Memory and HTTP Request Tools (`$fromAI` dynamic parameter binding).
- **Intuitive Explanation**: Docker runs both the Python server and n8n together. Inside n8n, an AI Agent node powered by GPT-4o-mini reads your chat message, decides which Python endpoint to call (`search_products`, `recommend_price`, or `simulate_price`), chains multiple calls together if needed, and writes a clear business summary.
- **CS/AI Example**: Functionally identical to the ReAct (Reason + Act) loop in agent frameworks like LangGraph, where an LLM iteratively emits tool-call ASTs, observes environment stdout, and synthesizes a final response.

---

3. Deep Dive: Stage 1 — Data Engineering & Demand Modeling (`src/1_prep_and_train.py`)

3.1 Data Hygiene & Noise Removal

- **Dataset**: Online Retail II dataset (`data/online_retail_II.csv`), comprising **1,067,371 raw rows** of UK e-commerce transactions, retaining **1,027,135 cleaned rows** after dropping **40,236 noisy/bulk rows**.
- **Step 1 — Cancelled Invoices (`~df['Invoice'].astype(str).str.startswith('C')`)**:
  - *Technical*: Filters out credit notes and order reversals that introduce negative demand shocks uncorrelated with current unit price.
  - *Intuitive*: Invoices starting with 'C' are customer returns/cancellations. Leaving them in corrupts weekly sales counts.
- **Step 2 — Non-Positive Filtering (`Quantity > 0` and `Price > 0`)**:
  - *Technical*: Eliminates accounting adjustments, damaged inventory write-offs, and complimentary promotional giveaways.
  - *Intuitive*: Keeps only genuine commercial purchases where a customer paid real money for positive units.
- **Step 3 — Non-SKU Exclusion (`['POST', 'M', 'BANK CHARGES', 'DOT', 'CRUK', 'PADS']`)**:
  - *Technical*: Removes service fees, shipping surcharges, and manual ledger entries that do not exhibit consumer price elasticity.
  - *Intuitive*: Postage (`POST`) and bank charges aren't physical retail products whose price you optimize for demand.
- **Step 4 — 99th Percentile Outlier Clipping (`df['Quantity'] <= df['Quantity'].quantile(0.99)`)**:
  - *Technical*: Truncates the upper 1% tail of order quantities to separate B2B wholesale bulk contracts from B2C retail demand distributions, preventing high-leverage outliers from skewing tree split gains under L2 loss.
  - *Intuitive*: Occasionally a corporate buyer orders 10,000 units at once. If we don't filter the top 1% of massive orders, the model will think a random week's price caused a 10,000-unit spike.
  - *CS/AI Example*: Analogous to removing DDoS traffic spikes or synthetic benchmark bursts when training an autoscaling load predictor for Kubernetes pods.

3.2 Feature Engineering & Weekly Aggregation (`aggregate_demand_features`)

- **Top-100 SKU Selection (`nlargest(100)`)**:
  - *Technical*: Restricts the panel to the top 100 SKUs by cumulative unit volume (~2.02 million total units), ensuring dense temporal coverage and sufficient price variance per cross-sectional unit. This yields **9,328 weekly demand observations**.

    - *Intuitive*: Products that only sold 3 times in two years don't have enough data to learn a price curve. Focusing on the top 100 bestsellers guarantees rich historical data.
    - *CS/AI Example*: Like vocabulary pruning in NLP tokenization (keeping the top 50k tokens with sufficient corpus frequency and dropping rare hapax legomena that would overfit embeddings).
- **Weekly Temporal Bucketing (`pd.Grouper(key='InvoiceDate', freq='W-MON')`)**:
    - *Technical*: Aggregates irregular point-process transaction timestamps into discrete weekly time series (`W-MON`), smoothing intra-week variance (e.g., weekend vs. weekday shopping gaps) and mitigating zero-inflation.
    - *Intuitive*: Individual transactions happen at random seconds. Grouping by week gives us a steady "units sold per week" number to predict.
- **Volume-Weighted Average Price (VWAP)**:
    - *Formula*: `price = total_revenue / total_quantity`
    - *Technical*: Computes the effective transaction-weighted price per period rather than an unweighted arithmetic mean of invoice lines, accurately weighting price points by actual consumer uptake.
    - *Intuitive*: If 100 units sell at £1.00 and 1 unit sells at £2.00 in the same week, an unweighted average says the price was £1.50—which is false because 99% of buyers paid £1.00. Dividing total weekly revenue (£102) by total quantity (101) gives the true effective price (£1.01).
    - *CS/AI Example*: Like computing weighted latency across microservices based on request throughput rather than averaging the latency of an idle endpoint and a high-traffic endpoint equally.
- **Relative Price Ratio (`price_ratio = price / ref_price`)**:
    - *Technical*: Normalizes nominal price against the SKU's historical median reference price (`ref_price`), creating a scale-invariant elasticity feature across heterogeneous price tiers (e.g., £0.21 keyrings vs. £12.58 cake stands).
    - *Intuitive*: A £0.50 change is huge for a £0.80 silk fan (+62%), but tiny for a £12.58 cake stand (+4%). `price_ratio` tells the model "this item is currently 15% above its normal price" regardless of whether it costs pennies or pounds.
    - *CS/AI Example*: Equivalent to Layer Normalization or Feature Standardization (z-score scaling) in neural networks so features with large magnitudes don't dominate smaller ones.
- **Calendar Seasonality (`month`, `week_of_year`, `quarter`)**:
    - *Technical*: Captures exogenous seasonal demand shifts (such as Q4 holiday gifting spikes) so the model does not confound seasonal volume surges with price elasticity.
    - *Intuitive*: Retail sales naturally jump in November and December. Adding calendar features stops the model from falsely thinking higher Q4 sales were caused by price changes.

- **Cost Estimation (`estimated_cost = round(median_price * 0.60, 2)`)**:
    - *Technical*: Imputes a standardized 60% Cost of Goods Sold (COGS) baseline (40% baseline gross margin) across the catalog to enable downstream gross profit optimization `((P - C) × Q)` when wholesale supplier invoices are absent from the public dataset.
    - *Intuitive*: The public UCI dataset includes retail selling prices but not what the store paid the factory. Setting unit cost at 60% of the normal selling price gives us a realistic 40% baseline margin to test profit optimization and margin floors.

3.3 Model Selection & Validation (`train_demand_model`)

- **Why LightGBM (`LGBMRegressor`)**:
    - *Technical*: Histogram-based Gradient Boosted Decision Trees (`n_estimators=150, learning_rate=0.05, num_leaves=31`) natively partition high-cardinality categorical variables (`StockCode` cast as `category`) using optimal Fisher exact sorting (O(k log k)) without one-hot encoding explosion, while capturing non-linear price thresholds and feature interactions (`price_ratio × quarter`).
    - *Intuitive*: If we used One-Hot Encoding for 100 products, we'd add 100 sparse columns. LightGBM handles categorical IDs natively, trains in under 2 seconds on a laptop CPU, and learns complex patterns like "discounts boost sales much more in Q4 than in Q1."
    - *CS/AI Example*: Like using a B-Tree index with range partitioning vs. a linear scan over sparse bitmaps.
- **Out-of-Time Validation Split (`split_date = max_date - 12 weeks`)**:
    - *Technical*: Enforces a strict temporal holdout split (reserving the final 12 weeks for testing) to prevent look-ahead bias and data leakage inherent in k-fold random cross-validation on autocorrelated time series.
    - *Intuitive*: If you shuffle time-series data randomly, the model gets to see next week's sales while predicting last week's sales—which is cheating. Holding out the final 12 weeks tests how well the model predicts the future.
    - *Metrics Achieved*: **Validation MAE = 133.48 units/week**, **R^2 = 0.159**. In noisy empirical retail data without promotional ad-spend logs or competitor prices, an out-of-time R^2 of ~0.16 on raw weekly volume across 100 diverse SKUs provides a solid signal-to-noise ratio for directional price optimization.

---

4. Deep Dive: Stage 2 — Constrained Optimization & Business Guardrails (`src/2_solver_and_guardrails.py`)

4.1 Why Unconstrained ML Fails in Pricing

- **Technical Framing**: Predictive models minimize empirical prediction error (MSE/L2 loss) over observed support but lack structural economic constraints. Extrapolating beyond the convex hull of training prices or optimizing purely for volume can yield negative unit margins or brand-damaging price volatility.
- **Intuitive Explanation**: Left unchecked, an ML model might say "charge £0.10 to sell 500 units" (losing money on every unit sold) or "double the price overnight" (scaring away loyal customers). Guardrails lock the optimizer inside a safe operating box.
- **CS/AI Example**: Just like an RL agent exploiting a physics simulator bug to get infinite reward unless you bound the action space (action clipping in PPO).

4.2 The Three Business Guardrails (`PricingGuardrails.compute_feasible_price_range`)

1. **Margin Floor (`min_margin_pct = 0.15`)**:
   - *Equation*: `margin_floor = unit_cost / (1.0 - min_margin_pct)`
   - *Technical*: Enforces a hard lower bound on gross margin percentage: `(P - C) / P ≥ 0.15` ⟺ `P ≥ C / 0.85`.
   - *Intuitive*: For Silk Fan `15036` (`unit_cost = £0.52`), `£0.52 / 0.85 = £0.6118`. Even if a deeper discount would increase unit volume, the system is mathematically forbidden from ever recommending a price below £0.61.
2. **Price Velocity Cap (`max_price_change_pct = 0.15`)**:
   - *Equation*: `velocity_min = base_price × 0.85`, `velocity_max = base_price × 1.15`
   - *Technical*: Implements a trust-region step constraint limiting single-period price adjustments to ±15% of the current reference price (`base_price`).
   - *Intuitive*: Prevents sticker shock. For `15036` (`base_price = £0.86`), the price in this cycle cannot drop below `£0.73` (`0.86 × 0.85`) or rise above `£0.99` (`0.86 × 1.15`).
3. **Historical Support Corridor (`min_hist_price × 0.9` to `max_hist_price × 1.1`)**:
   - *Technical*: Restricts inference to within ±10% of the empirical training support `[min_observed_price, max_observed_price]` to prevent out-of-distribution (OOD) tree extrapolation (since decision trees output constant leaf values outside their outermost split thresholds).
   - *Intuitive*: Decision trees cannot extrapolate trends outside the prices they saw during training—they just flatline. Keeping prices near historical min/max ensures the model's demand predictions remain trustworthy.

4. **Intersection & Fallback Resolution**:
    - *Equation*:
        - `lower_bound = max(margin_floor, velocity_min, min_hist_price × 0.9)`
        - `upper_bound = min(velocity_max, max_hist_price × 1.1)`
    - *Fallback*: If `lower_bound > upper_bound` (infeasible intersection due to an extreme cost spike), both bounds gracefully collapse to `base_price` so the solver never crashes.

4.3 Why 1D Discretized Grid Search Instead of SciPy Gradient Descent (`optimize_price`)

- **Technical Framing**: Tree-based ensembles (`LGBMRegressor`) define a piecewise-constant step function `Q(P)`. Consequently, the gradient `∂Q/∂P` is zero almost everywhere and undefined at split thresholds. Gradient-based solvers (L-BFGS, SLSQP, Newton-Raphson) immediately terminate at the initial point with zero gradient. A 50-point uniform grid (`np.linspace(p_min, p_max, 50)`) over the compact 1D interval `[p_min, p_max]` guarantees finding the global maximum on the grid in O(N) time (~5ms).
- **Intuitive Explanation**: Calculus-based optimizers work by "walking uphill" using the slope of a smooth curve. Because decision trees work like staircases (flat steps with sudden drops), the slope on any flat step is zero, so a calculus solver thinks it's already at the top and gets stuck! Testing 50 evenly spaced prices across our narrow allowed window (`£0.73` to `£0.99`) checks every step on the staircase in a few milliseconds and is guaranteed to find the true highest profit.
- **CS/AI Example**: Like searching an array of discrete quantized step values using a linear scan rather than trying to run gradient descent over a step-activation function (like a perceptron heaviside step function).

4.4 Concrete Walkthrough: StockCode `15036` ("ASSORTED COLOURS SILK FAN")

- **Inputs**: `base_price = £0.86`, `unit_cost = £0.52`, `min_observed_price = £0.63`, `max_observed_price = £1.63`.
- **Guardrail Corridor Calculation**:
    - `margin_floor = 0.52 / 0.85 = £0.61`
    - `velocity_min = 0.86 × 0.85 = £0.731`
    - `velocity_max = 0.86 × 1.15 = £0.989`
    - `lower_bound = max(0.61, 0.731, 0.567) = £0.73`
    - `upper_bound = min(0.989, 1.793) = £0.99`
- **Baseline Performance (at £0.86)**:
    - Projected Demand: `125.2 units/week`

    - Projected Revenue: `£107.66`
    - Projected Gross Profit: `(0.86 - 0.52) × 125.2 = £42.56`
- **Optimal Recommendation (`max_profit` at £0.99)**:
    - Recommended Price: `£0.99` (+15.1% vs. baseline)
    - Projected Demand: `128.6 units/week`
    - Projected Revenue: `£127.31` (+18.3% revenue lift)
    - Projected Gross Profit: `£60.44` (**+42.0% profit lift**)
- **Discount Simulation (`proposed_price = £0.50`)**:
    - Projected Demand jumps to `168.2 units/week`, but because `£0.50 < £0.52 unit_cost`, Gross Profit drops to **-£3.36 (-4.0% margin)** and `guardrail_status` returns **`FLAGGED`** (`Price £0.50 is below the allowed safety floor £0.73`).

---

5. Deep Dive: Stage 3 — FastAPI Service & LLM Resilience (`src/3_api_and_agent.py`)

5.1 Singleton Model Loading on Startup

- **Technical Framing**: Instantiates `optimizer = PricingOptimizer()` at module import scope so the LightGBM booster and product catalog dictionary reside in resident memory across worker requests, achieving sub-10ms inference latency.
- **Intuitive Explanation**: Instead of loading the model from the hard drive every time someone asks a question, FastAPI loads it once when the server starts and reuses it for every request.

5.2 Resilient Input Sanitization (`@field_validator(mode="before")`)

- **The Real-World Bug We Solved**: During testing, when the user asked *"What happens if we discount it to £0.50?"*, the LLM passed `"£0.50"` into the tool call. Similarly, when asked to *"maximize our gross profit"*, the LLM passed `"maximize our gross profit"` instead of the strict enum `"max_profit"`. Standard Pydantic validation threw `422 Unprocessable Entity` errors.
- **Technical Solution**:
    - `clean_price`: Uses regex `re.sub(r"[^\d.]", "", str(v))` inside a `mode="before"` validator to strip currency symbols (£, $), commas, and whitespace before float casting.

    - `normalize_objective`: Inspects lowercase string substrings (`"max_revenue" if "rev" in val else "max_profit"`), gracefully mapping any natural-language variation of profit or revenue to the exact solver constant.
    - `clean_stock_code`: Coerces integer SKU codes (`15036`) and padded strings (`" 15036 "`) into canonical strings.
- **CS/AI Example**: This is a classic implementation of **Postel's Law (the Robustness Principle)** in distributed systems and agentic tool design: *"Be conservative in what you send, be liberal in what you accept."*

5.3 Supporting Both `GET` (Query Params) and `POST` (JSON Body)

- **Technical Framing**: Exposes dual `GET` and `POST` handlers for `/pricing/recommend` and `/pricing/simulate`. While REST conventions favor `POST` for complex payloads, LangChain/n8n `toolHttpRequest` nodes bind `$fromAI()` expressions into URL query parameters (`parametersQuery`) with zero JSON serialization or quote-escaping failures.
- **Intuitive Explanation**: Having both `GET` and `POST` means our API works seamlessly whether called by a traditional web frontend sending JSON (`POST`) or an n8n AI agent passing URL parameters (`GET`).

---

6. Deep Dive: Stage 4 — Agentic Orchestration & Docker Stack (n8n + Docker)

6.1 How the n8n AI Agent Works (`n8n_pricing_workflow.json`)

- **Architecture**:
    a. **Chat Trigger (`@n8n/n8n-nodes-langchain.chatTrigger`)**: Captures user prompts in the n8n UI.
    b. **AI Pricing Agent (`@n8n/n8n-nodes-langchain.agent`)**: Configured with a structured System Prompt instructing the agent on tool routing rules (search first for keywords, optimize for recommendations, simulate for what-if prices).
    c. **OpenAI Chat Model (`gpt-4o-mini`, `temperature = 0.2`)**: Low temperature ensures deterministic, analytical tool selection and prevents creative exaggeration of financial figures.
    d. **Window Buffer Memory (`contextWindowLength = 10`)**: Maintains the last 10 turns of conversation state so follow-up pronouns (e.g., *"What if we discount **it** to £0.50?"*) resolve to the previously discussed `StockCode` (`15036`).
    e. **Three HTTP Request Tools (`$fromAI` Dynamic Binding)**:

        - `search_products`: `GET /products?search={{ $fromAI('search', ...) }}&limit=10`
        - `recommend_price`: `GET /pricing/recommend?stock_code={{ $fromAI('stock_code', ...) }}&objective={{ $fromAI('objective', ...) }}`
        - `simulate_price`: `GET /pricing/simulate?stock_code={{ $fromAI('stock_code', ...) }}&proposed_price={{ $fromAI('proposed_price', ...) }}`

6.2 Containerization & Networking (`Dockerfile` & `docker-compose.yml`)

- **Why `libgomp1` in the `Dockerfile`:**
    - *Technical*: We use `python:3.11-slim` (a minimal Debian bookworm image) to keep the container lightweight. LightGBM's C++ core is compiled with OpenMP multi-threading support, which dynamically links against `libgomp.so.1`. Installing `libgomp1` via `apt-get` prevents `OSError: libgomp.so.1: cannot open shared object file` at container startup.
- **Docker Networking (`host.docker.internal` & Service DNS):**
    - *Technical*: Inside a Docker container, `localhost` (`127.0.0.1`) resolves to the container's own isolated network namespace. Binding FastAPI to `0.0.0.0:8000` and routing via `host.docker.internal:8000` (or the Docker Compose service name `pricing-api:8000`) bridges the container network to the FastAPI server.
- **Persistent Volume (`n8n_data:/home/node/.n8n`):**
    - *Technical*: Mounts a named Docker volume to persist n8n's SQLite database, encrypted API credentials, and workflow definitions across `docker compose down` / `up` lifecycles.

---

7. Real Engineering Challenges We Debugged (High-Value Interview Stories)

Use these four stories when asked *"Tell me about a technical challenge you faced while building this"*:

1. **The Python Module Numeric Prefix Bug (`SyntaxError: invalid decimal literal`):**
    - *Symptom*: Importing `from src.2_solver_and_guardrails import PricingOptimizer` threw a Python syntax error.

- *Root Cause*: Python's lexical scanner prohibits identifiers starting with digits so the parser can distinguish variable names from numeric literals in a single lookahead token.
- *Fix*: Used `importlib.import_module("src.2_solver_and_guardrails")` to load the module programmatically at runtime without breaking the numbered pedagogical file structure.

2. **The Shell Variable Expansion Bug in `$fromAI` (`n8n_pricing_workflow.json`)**:
   - *Symptom*: n8n sent the literal description text `"Product StockCode, e.g. 15036"` as the `stock_code` parameter instead of `"15036"`.
   - *Root Cause*: Generating the JSON inside a double-quoted bash string caused bash to interpret `$fromAI` as an undefined shell environment variable, replacing it with an empty string and leaving `{{ ("stock_code", "Product StockCode, e.g. 15036") }}`—which JavaScript evaluated using the comma operator (returning the last operand).
   - *Fix*: Wrote the JSON directly via file I/O without shell interpolation, preserving the `$fromAI(...)` expression intact.
3. **LLM Type-Rigidity & Currency Symbol Failures (`422 Unprocessable Entity`)**:
   - *Symptom*: The LLM passed `"£0.50"` for price and `"maximize gross profit"` for objective, failing strict Pydantic float/Literal validation.
   - *Fix*: Implemented Pydantic `@field_validator(mode="before")` sanitizers and `GET` query parameter endpoints, making the API resilient to natural-language variations.
4. **Why Demand Slightly Increased at £0.99 vs £0.86 for SKU `15036`**:
   - *Observation*: At £0.86 predicted demand is 125.2 units, while at £0.99 it is 128.6 units.
   - *Technical Explanation*: In observational retail data without explicit promotional flags, tree models split on historical price clusters where higher prices coincided with peak Q4 gift demand (endogeneity / confounding). Standard LightGBM does not enforce monotonic constraints by default.
   - *How to Answer in the Interview*: *"Because standard GBDTs split greedily on empirical correlations, seasonal demand surges can create local non-monotonic steps in the demand curve. In V2, passing `monotone_constraints=[0, -1, 0, -1, 0, 0, 0]` to `LGBMRegressor` mathematically guarantees that ∂Q/∂P ≤ 0 everywhere (predicted demand strictly decreases or stays flat as price increases)."*

---

8. High-Probability Interview Questions & Model Answers

Q1: "Why didn't you just give the CSV data to an LLM and ask it to recommend prices?"

Model Answer: "LLMs are probabilistic next-token predictors, not deterministic math engines. Feeding 1 million transaction rows into an LLM is impossible due to context-window limits and token costs, and LLMs routinely hallucinate arithmetic and elasticity curves. By decoupling the architecture—using LightGBM for statistical demand estimation, a deterministic Python solver for constrained optimization, and the LLM strictly for semantic intent parsing and tool orchestration—we get 100% mathematical accuracy, hard safety guardrails, and sub-second latency."

Q2: "Why did you choose LightGBM over Linear Regression or a Deep Neural Network?"

Model Answer: "Linear regression assumes constant price elasticity across the entire price range and requires manual one-hot encoding for every SKU. Deep neural networks require massive hyperparameter tuning and struggle with tabular data containing skewed distributions and categorical IDs. LightGBM natively partitions categorical StockCode features using histogram binning, captures non-linear price thresholds and seasonal interactions out of the box, trains on 9,300 weekly rows in under 2 seconds on a CPU, and provides fast inference."

Q3: "Your test R^2 is 0.159. Is that good enough to make pricing decisions?"

Model Answer: "In real-world retail panel data evaluated on a strict out-of-time holdout (the final 12 weeks) without external marketing spend, competitor pricing, or inventory stockout logs, weekly item volume has high natural variance. An out-of-time R^2 of 0.16 with a 133-unit MAE captures the core price-seasonality signal without overfitting. More importantly, because our predictive model is wrapped inside deterministic business guardrails—a 15% margin floor and a ±15% velocity corridor—even when demand predictions have variance, the optimizer can never recommend a destructive or unprofitable price."

Q4: "How would you evolve this MVP for a production enterprise deployment?"

Model Answer:

1. **Monotonicity Constraints**: Add `monotone_constraints` in LightGBM on `price` and `price_ratio` (`-1`) so demand is strictly non-increasing with price.
2. **Cross-Price Elasticity (Cannibalization)**: Add category-level average price features so discounting one Red Spotty Lunch Bag accounts for cannibalizing sales from the Woodland Lunch Bag.

3. **Contextual Bandits / Thompson Sampling**: Replace static grid search with an explore-exploit bandit framework that actively tests price points with high epistemic uncertainty.
4. **Human-in-the-Loop (HITL) Approval Workflow**: Add a Slack/Teams approval node in n8n so any price recommendation changing price by >5% requires one-click manager approval before writing back to the ERP/Shopify database.